

Deep Learning (Fall 2025)

Homework 2

Student ID: 314505013

Name: 詹炘達

1. MNIST:

NETWORK:

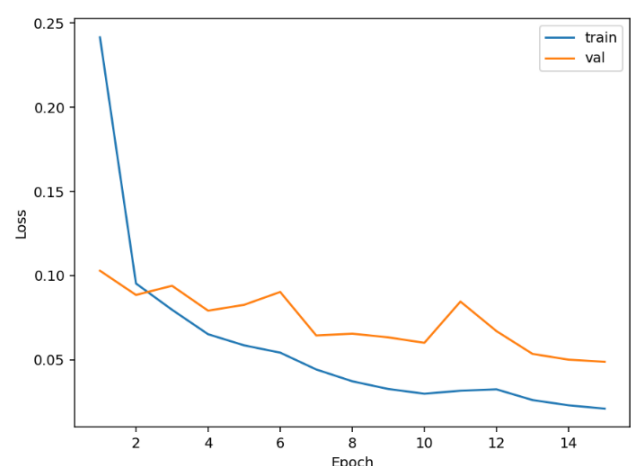
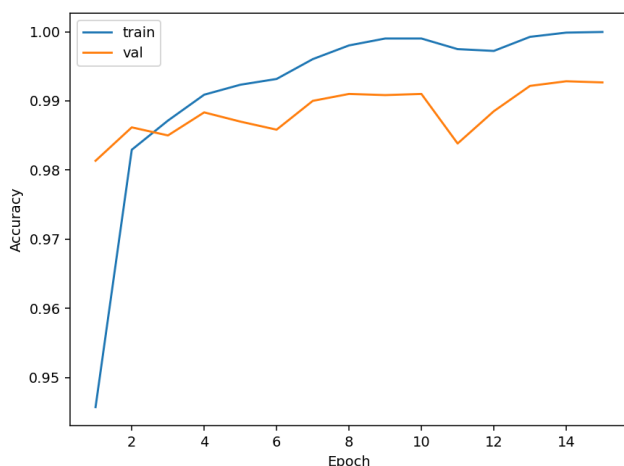
```
42 def build_model(kernel=3, stride=1, l2=0.0, dropout=0.3):
43     reg = regularizers.l2(l2) if l2>0 else None
44     he = initializers.HeNormal()
45
46     inputs = layers.Input(shape=(28,28,1))
47     x = layers.Conv2D(32, kernel_size=kernel, strides=stride, padding="same",
48                       activation=None, use_bias=False, kernel_initializer=he,
49                       kernel_regularizer=reg)(inputs)
50     x = layers.BatchNormalization()(x); x = layers.ReLU()(x)
51     x = layers.MaxPool2D(2)(x)
52
53     x = layers.Conv2D(64, kernel_size=kernel, strides=1, padding="same",
54                       activation=None, use_bias=False, kernel_initializer=he,
55                       kernel_regularizer=reg)(x)
56     x = layers.BatchNormalization()(x); x = layers.ReLU()(x)
57     x = layers.MaxPool2D(2)(x)
58
59     x = layers.Flatten()(x)
60     x = layers.Dense(128, activation="relu", kernel_initializer=he,
61                     kernel_regularizer=reg)(x)
62     if dropout and dropout>0:
63         x = layers.Dropout(dropout)(x)
64     outputs = layers.Dense(num_classes, activation="softmax", kernel_initializer="glorot_uniform")(x)
65
66     opt = optimizers.AdamW(learning_rate=1e-3, weight_decay=l2 if l2>0 else 0.0)
67     model = tf.keras.Model(inputs, outputs)
68     model.compile(optimizer=opt, loss="sparse_categorical_crossentropy", metrics=["accuracy"])
69     return model
70
```

Designed Parameter (the later discussion will be made according to this design):

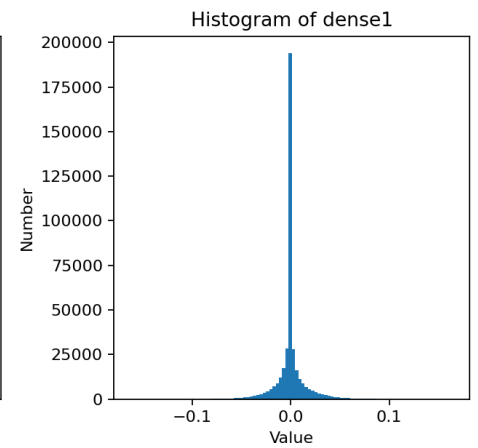
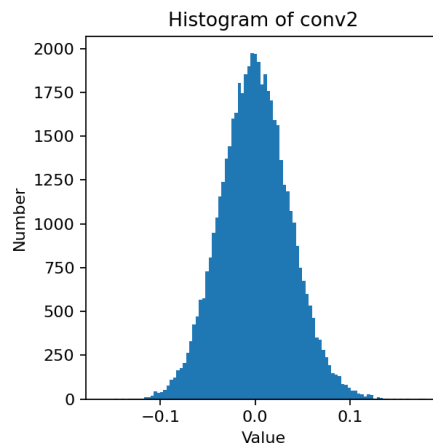
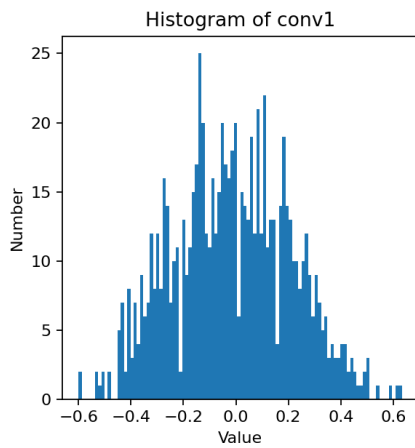
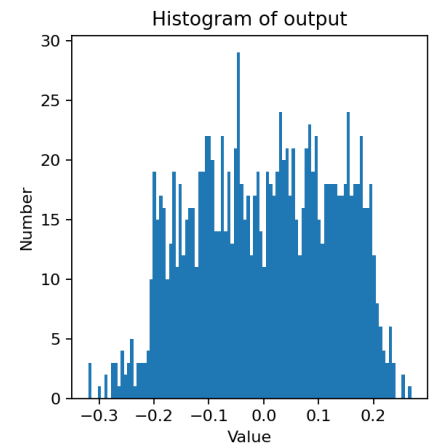
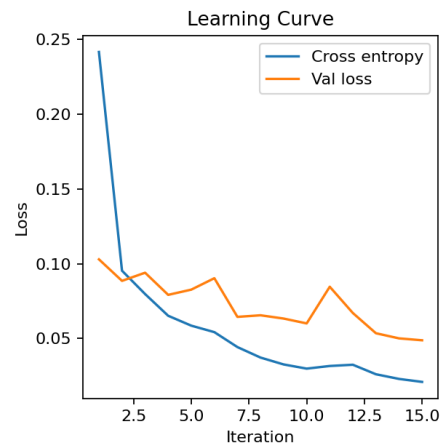
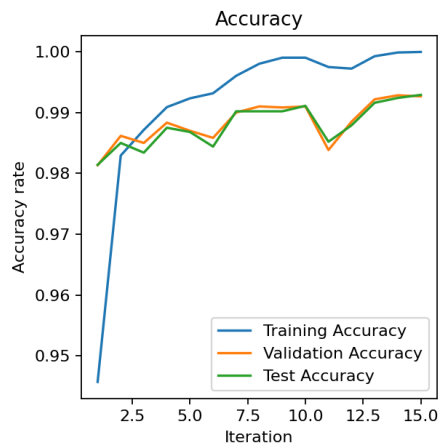
Kernel	5*5
Stride	1
L2 normalization	1e-4
Dropout	0

Test & Val Accuracy: 0.999 & 0.993

Test & Val Loss: 0.02104 & 0.0489



Accuracy, learning curve, and histograms of layers

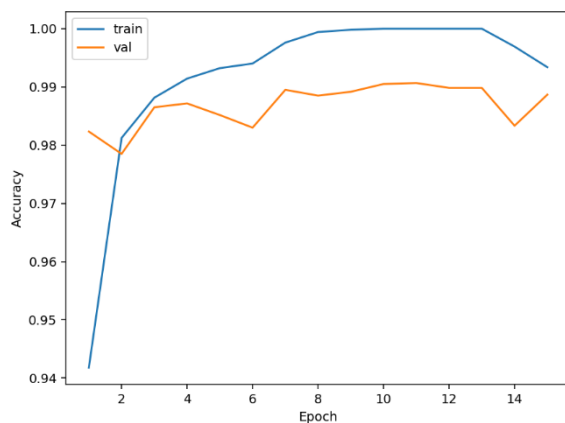


1-1. Comparison of Different Stride Sizes and Filter Size: (Only 3 for examples, other setups can be seen in the appended code):

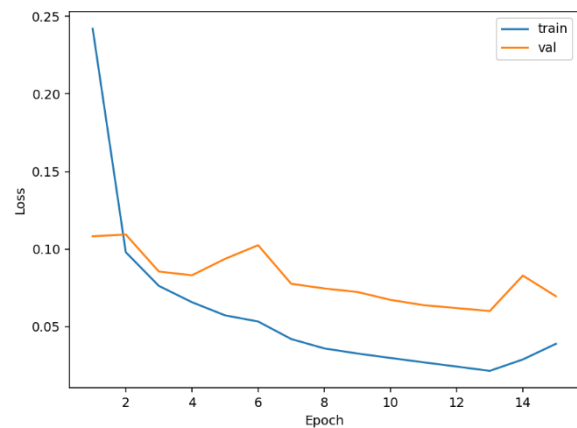
Kernel = 5*5, Stride = 2:

Kernel	5*5
Stride	2
L2 normalization	1e-4
Dropout	0

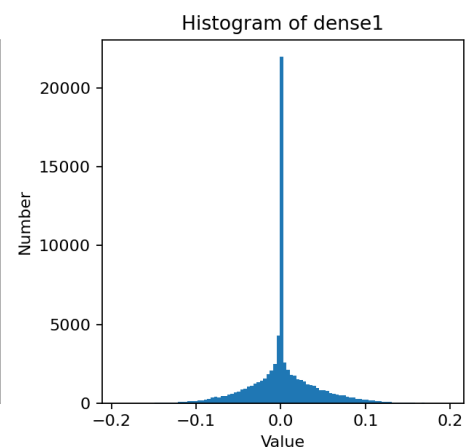
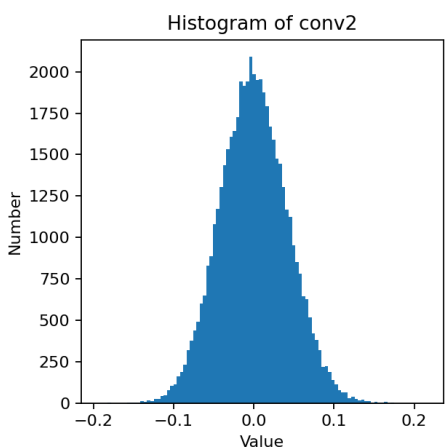
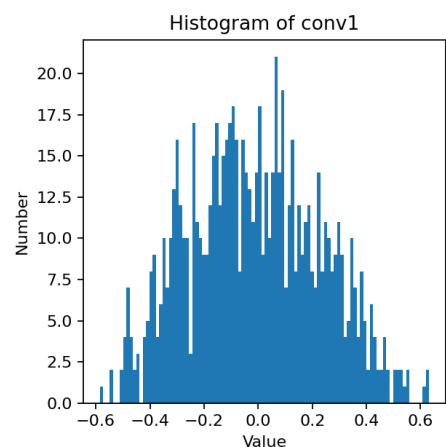
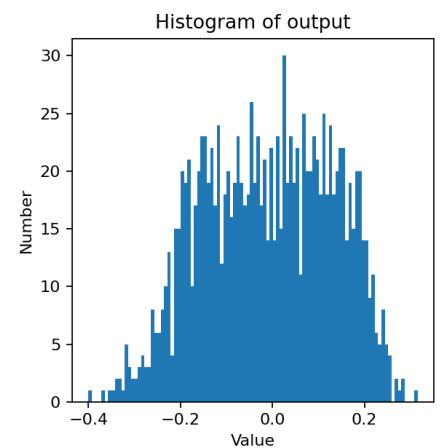
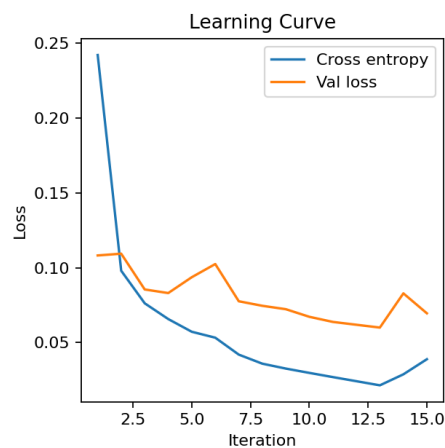
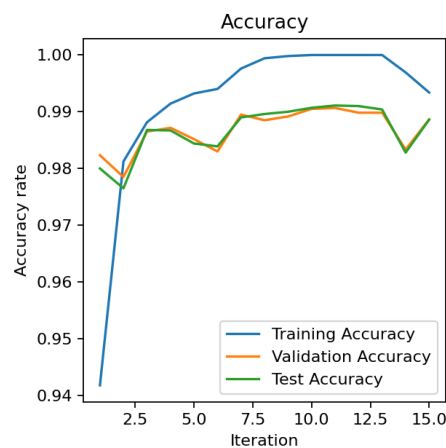
Test & Val Accuracy: 0.999 & 0.992



Test & Val Loss: 0.0210 & 0.0488



Accuracy, learning curve, and histograms of layers

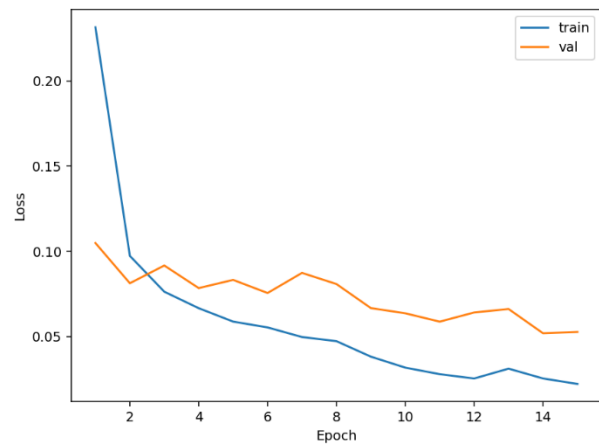
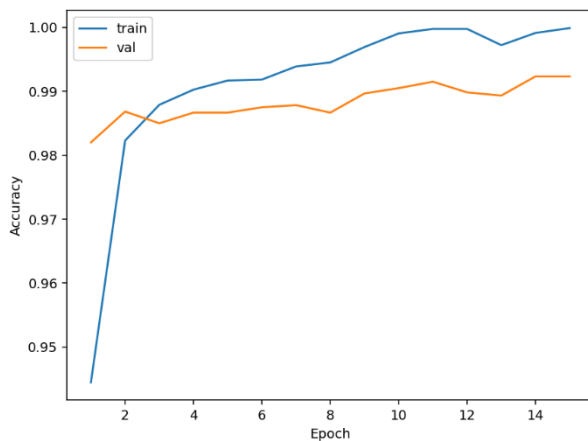


Kernel = 3*3, Stride = 1:

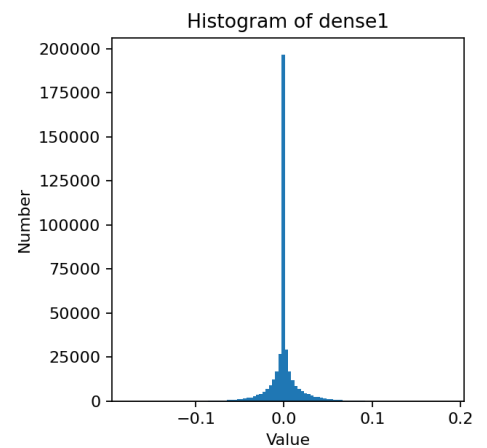
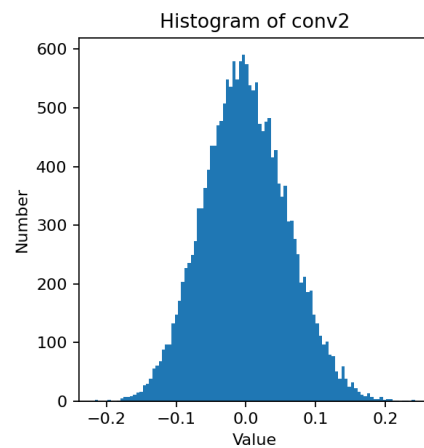
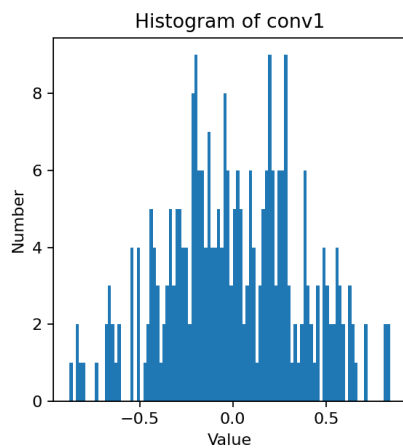
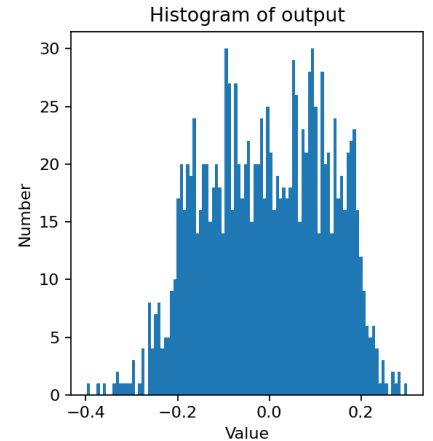
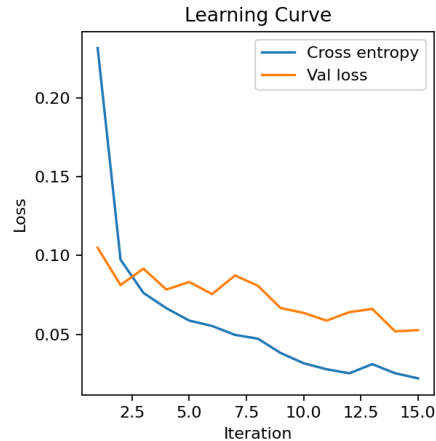
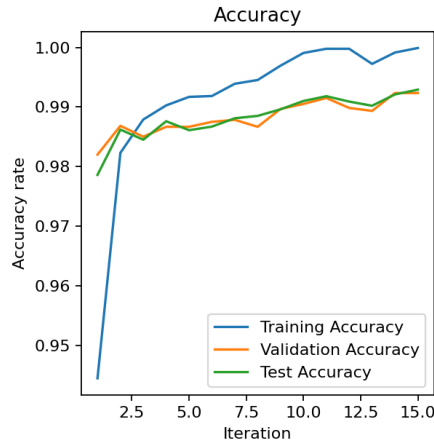
Kernel	3*3
Stride	1
L2 normalization	1e-4
Dropout	0

Test & Val Accuracy: 0.999 & 0.992

Test & Val Loss: .0222 & 0.0527



Accuracy, learning curve, and histograms of layers

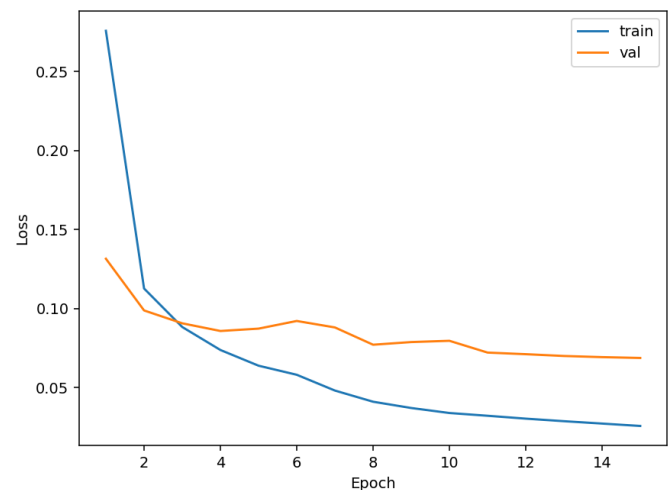
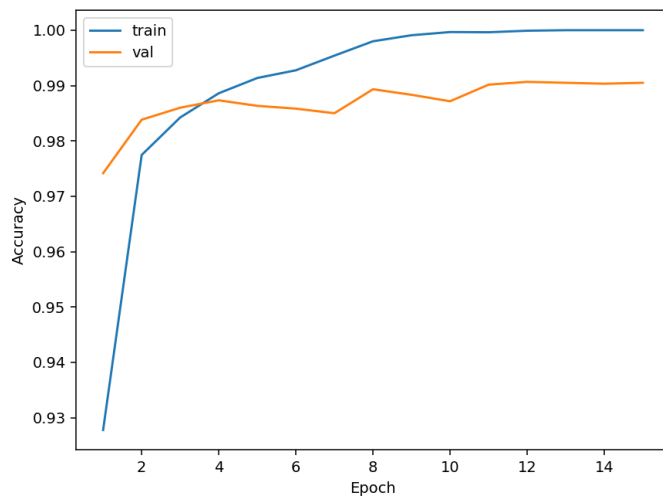


Kernel =3*3, Stride = 2:

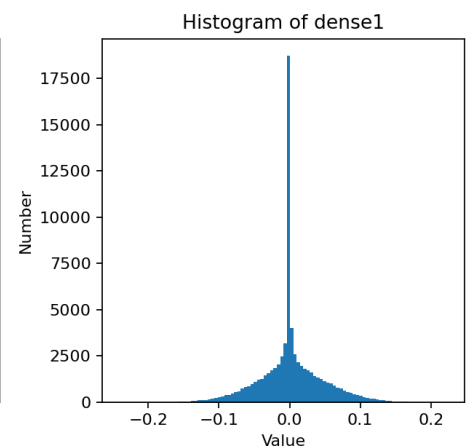
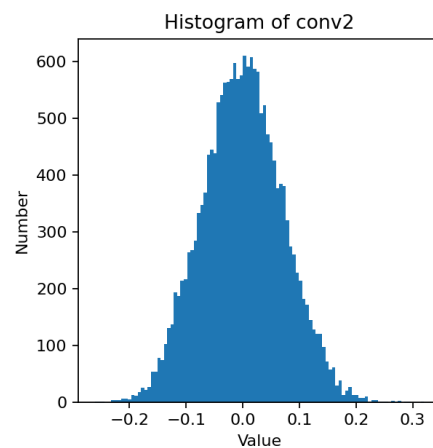
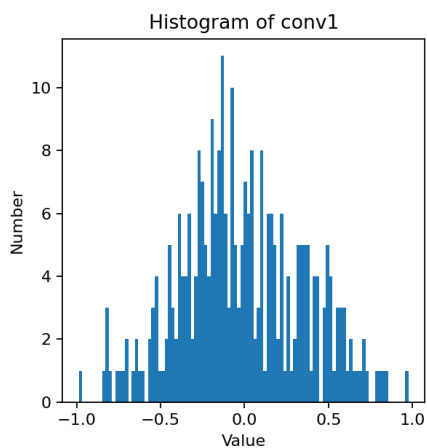
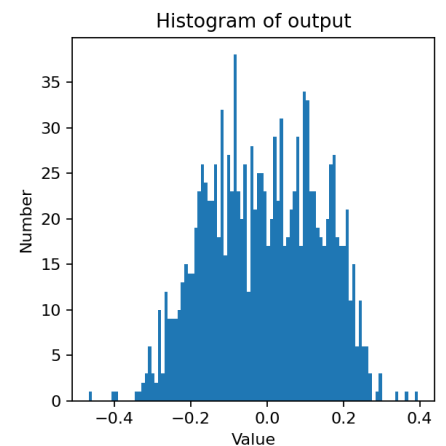
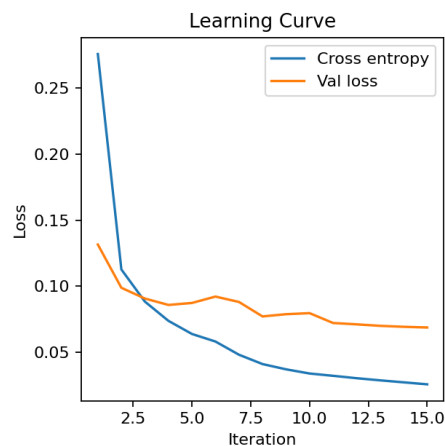
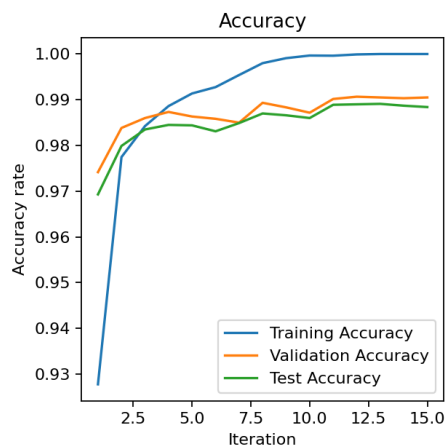
Kernel	3*3
Stride	2
L2 normalization	1e-4
Dropout	0

Test & Val Accuracy: 0.9998 &0.992

Test & Val Loss: 0.02104 & 0.0489



Accuracy, learning curve, and histograms of layers



Analysis:

1. Comparison of Kernel Size (Filter Size)

On relatively simple datasets like MNIST, using a 5×5 kernel slightly improves test accuracy compared to a 3×3 kernel. Specifically:

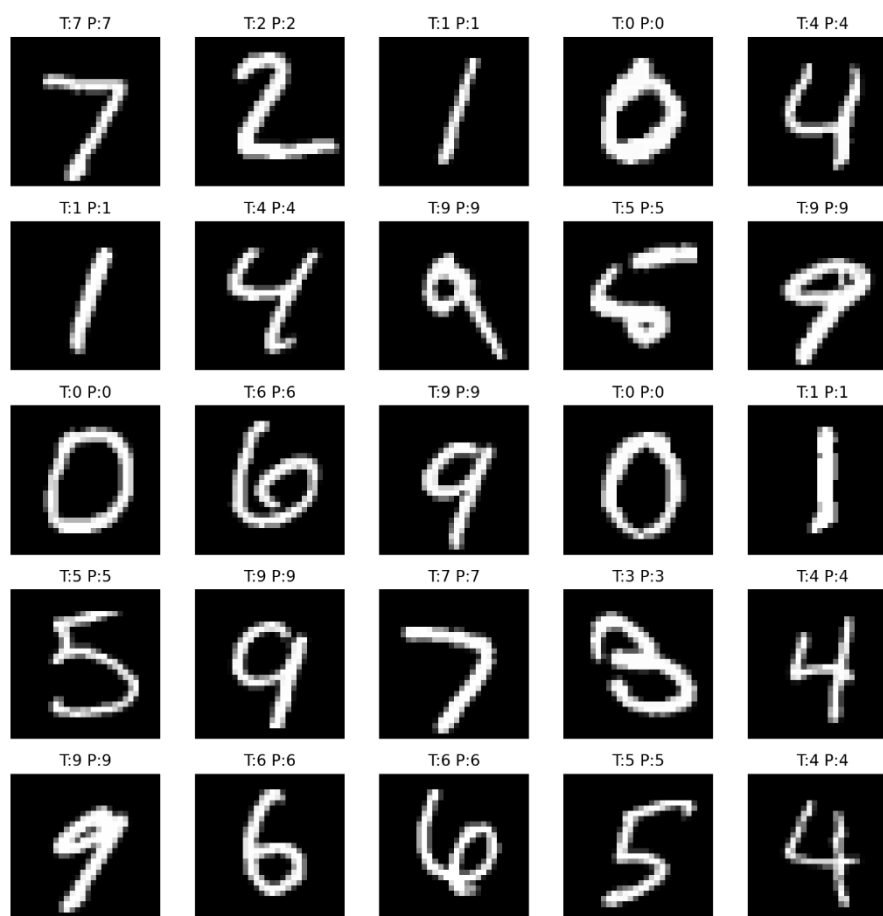
- **5×5 Kernel:** The test accuracy is around 99.2%, slightly better than the 99.0% of the 3×3 kernel. A larger kernel can detect larger regional features in the image. A larger receptive field helps improve recognition accuracy for simple handwritten digit classification tasks. When the kernel size increases, it can capture more extensive features from the image, such as the overall structure of the strokes.
- **3×3 Kernel:** While the performance is slightly lower, 99.0% accuracy is still very high, and the computational cost is much smaller. For simpler datasets like MNIST, this kernel size is already sufficient to achieve good results.

2. Comparison of Stride

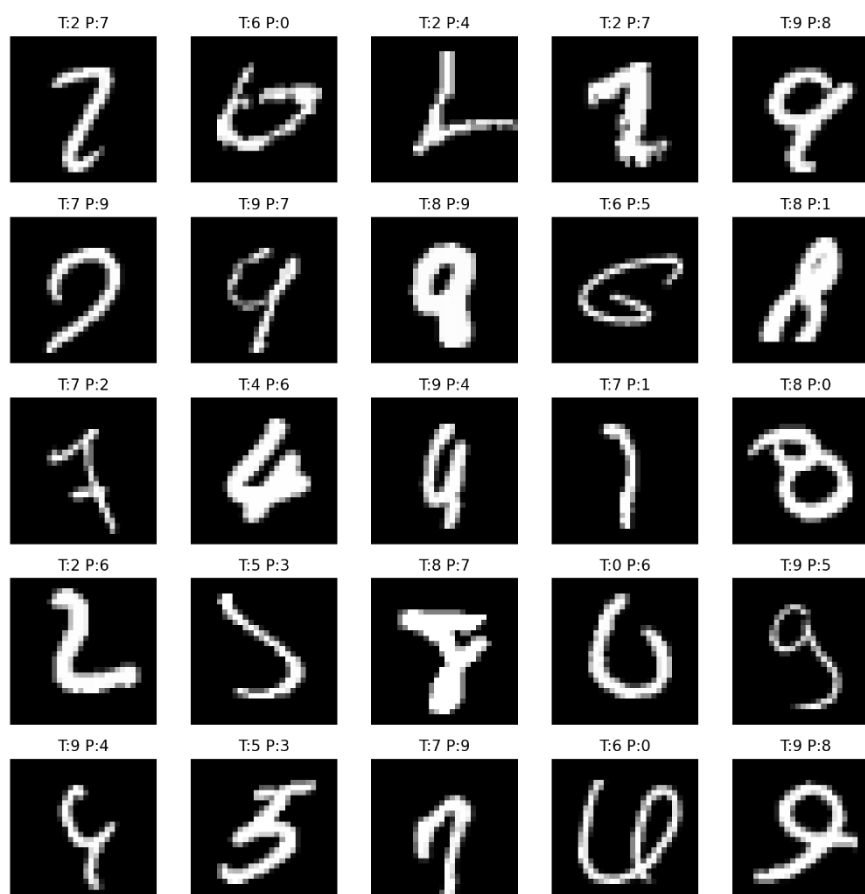
- **Stride = 1:** When the stride is set to 1, the convolutional kernel moves 1 pixel at a time, allowing it to retain more image details. This is why models with stride = 1 usually have higher accuracy (e.g., 99.0% or 99.2%). It can capture finer details in the image, which helps distinguish subtle differences between digits.
- **Stride = 2:** When the stride is set to 2, the resolution of the feature map decreases, which leads to the loss of some details in the image. Therefore, models with stride = 2 tend to have a lower test accuracy on MNIST (around 98.5% or 98.8%). However, on more complex datasets like CIFAR-10, increasing the stride can help reduce the computational load and speed up training.

1-2. Examples of correctly classified and misclassified images

Correctly Classified Image:



Misclassified Images:



Discussion of the Two Images:

1. Correctly Classified Samples

- We can observe that the model performs well in clear, well-formed digits. For example, the predictions for "7", "1", "4", and "9" are all correct, which shows that the model is effectively learning the features of clearly written digits.
- The model excels at identifying digits that are written clearly and follow standard patterns. It performs exceptionally well on simpler, well-defined digits such as "1" and "7".

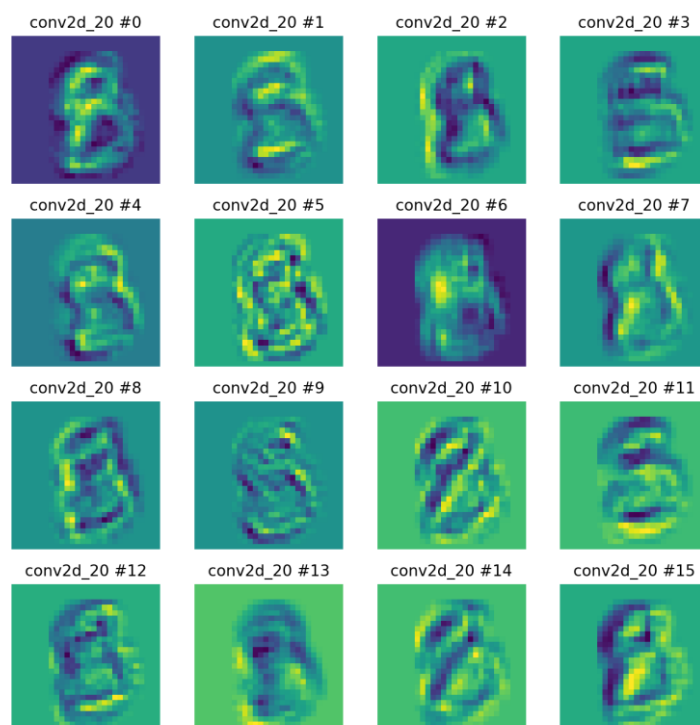
2. Misclassified Samples

- The errors often occur with digits that are visually similar or have slight distortions. For instance, "T: 2 P: 9" and "T: 4 P: 9" show how the model may confuse numbers with similar shapes, especially when the handwriting is unclear or distorted. The model struggles to differentiate between digits like "2" and "9" or "4" and "9" because they share some standard strokes and shapes.
- Errors seem to be more frequent when the digits are written in a non-standard way, or when there is overlap or ambiguity in the shapes of the digits. Misclassifications typically happen with numbers with similar visual features and are often due to variations in writing styles or slight distortions in the input images.

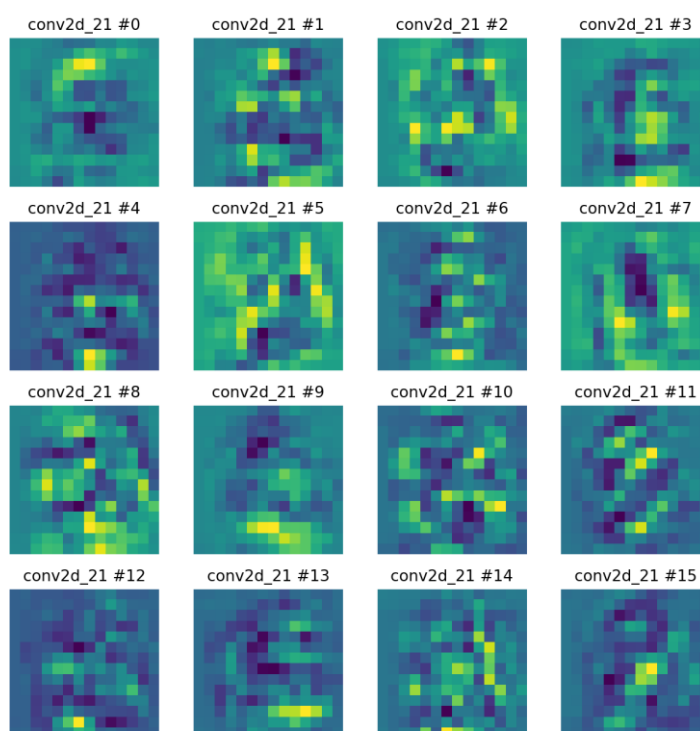
Conclusion:

- The model performs excellently in well-formed, standard handwritten digits.
- The model faces challenges when dealing with digits that have similar shapes or are written in less standard forms. Digits such as "2", "9", and "4" often get misclassified due to their visual similarity. More training data, including more varied handwriting styles, could help mitigate these errors and improve overall model robustness.

1-3. Feature maps from different convolutional layers



The *earlier layers* (*conv2d_20*) focus on simple features (edges, lines, and textures).



The *deeper layers* (*conv2d_21*) focus on more complex structures, which could be parts of objects, shapes, or even more intricate details necessary for higher-level tasks like classification.

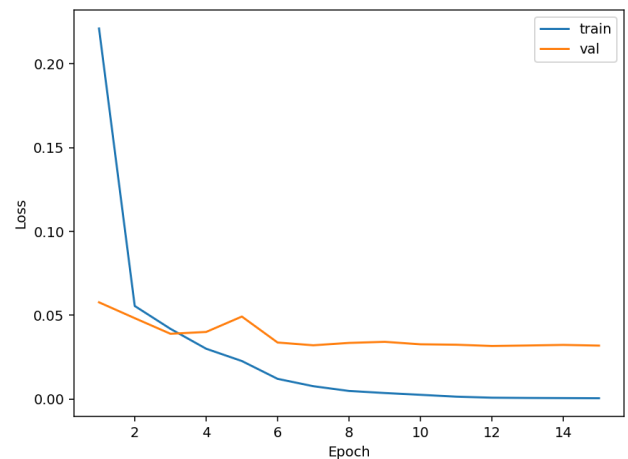
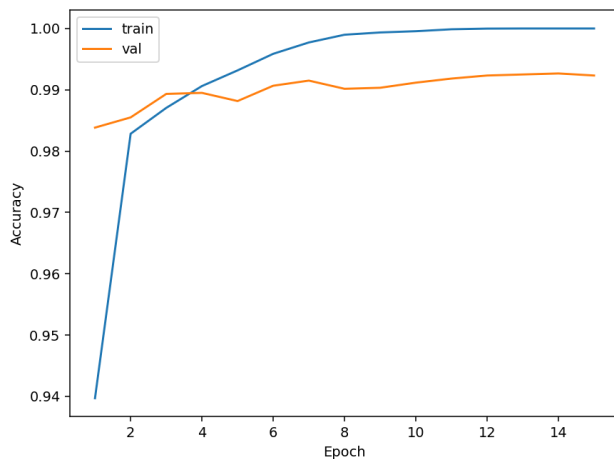
1-4. The effect of L2 regularization

Let L2 normalization = 0, and show the layers' Accuracy, learning curve, and histograms.

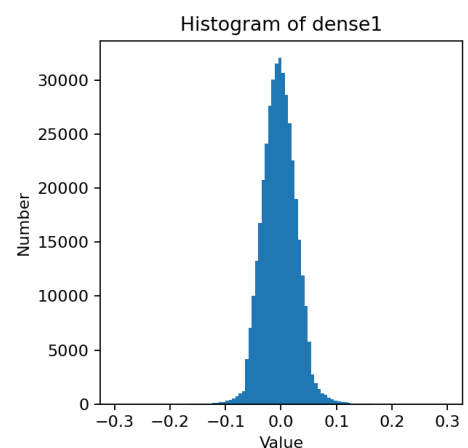
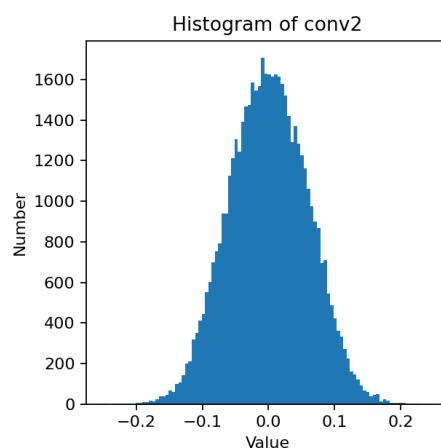
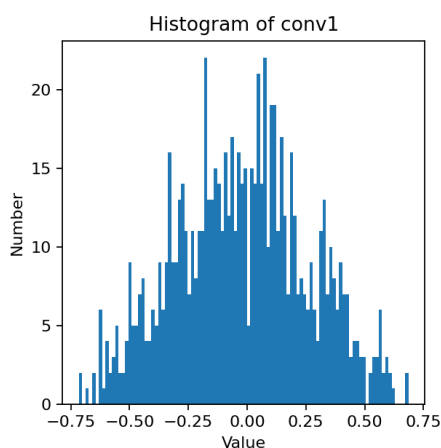
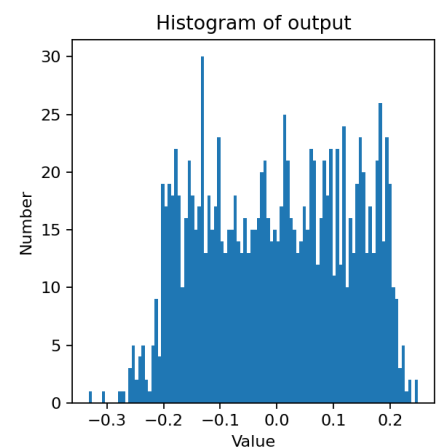
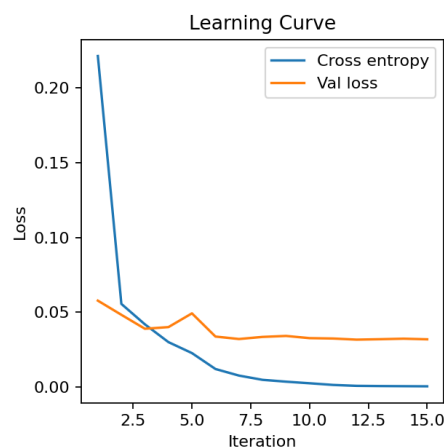
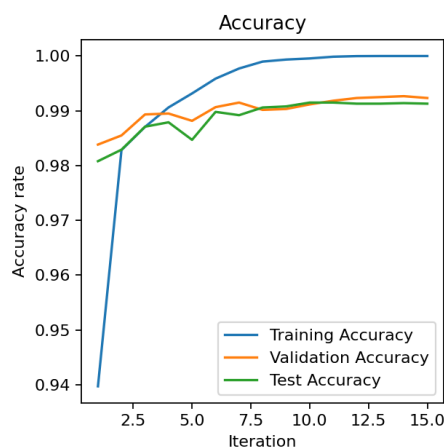
Kernel	5*5
Stride	1
L2 normalization	0
Dropout	0

Test & Val Accuracy: 1 & 0.992

Test & Val Loss: 0.0005 & 0.031



Accuracy, learning curve, and histograms of layers



Discussion about Regularization

Condition	Test Accuracy	Validation Accuracy	Test Loss	Validation Loss
With L2 ($1e-4$)	0.999	0.993	0.0005	0.031
Without L2 (0)	1.000	0.992	0.021	0.0489

Learning Behavior

- Without L2 regularization, the training accuracy shows a very steep initial rise and converges fast.
- With L2 regularization, the learning curves are slightly smoother and show slower convergence. The gap between training and validation accuracy is smaller, suggesting enhanced model generalization.
 - ⇒ L2 stabilizes learning, reducing rapid overfitting.

Weight and Activation Distribution

- The weight histograms show that models trained with L2 have narrower and centered-near-0 weight distributions.
- In contrast, the non-regularized model exhibits a broader distribution.
 - ⇒ L2 constrains weight growth and smooths parameters.

Generalization Effect

- The performance of the model on the MNIST dataset is already great, but L2 adds little benefit.

2. CIFAR 10

Network:

```
51 def build_model(kernel=3, l2=5e-4, dropout=0.4, lr=1e-3, stride=1):
52     reg = regularizers.l2(l2) if l2>0 else None
53     he = initializers.HeNormal()
54     inputs = layers.Input(shape=(32,32,3))
55     x = augment()(inputs)
56
57     x = layers.Conv2D(32, kernel_size=kernel, strides=stride, padding="same",
58                       activation=None, use_bias=False, kernel_initializer=he, kernel_regularizer=reg)(x)
59     x = layers.BatchNormalization()(x); x = layers.ReLU()(x)
60     x = layers.Conv2D(32, kernel_size=kernel, strides=stride, padding="same",
61                       activation=None, use_bias=False, kernel_initializer=he, kernel_regularizer=reg)(x)
62     x = layers.BatchNormalization()(x); x = layers.ReLU()(x)
63     x = layers.MaxPool2D(2)(x)
64
65     x = layers.Conv2D(64, kernel_size=kernel, strides=1, padding="same",
66                       activation=None, use_bias=False, kernel_initializer=he, kernel_regularizer=reg)(x)
67     x = layers.BatchNormalization()(x); x = layers.ReLU()(x)
68     x = layers.Conv2D(64, kernel_size=kernel, strides=1, padding="same",
69                       activation=None, use_bias=False, kernel_initializer=he, kernel_regularizer=reg)(x)
70     x = layers.BatchNormalization()(x); x = layers.ReLU()(x)
71     x = layers.MaxPool2D(2)(x)
72
73     x = layers.Conv2D(128, kernel_size=kernel, strides=1, padding="same",
74                       activation=None, use_bias=False, kernel_initializer=he, kernel_regularizer=reg)(x)
75     x = layers.BatchNormalization()(x); x = layers.ReLU()(x)
76
77     x = layers.GlobalAveragePooling2D()(x)
78     x = layers.Dense(128, activation="relu", kernel_initializer=he, kernel_regularizer=reg)(x)
79     if dropout and dropout>0:
80         x = layers.Dropout(dropout)(x)
81     outputs = layers.Dense(10, activation="softmax", kernel_initializer="glorot_uniform")(x)
82
83     opt = optimizers.AdamW(learning_rate=lr, weight_decay=l2)
84     model = tf.keras.Model(inputs, outputs)
85     model.compile(optimizer=opt, loss="sparse_categorical_crossentropy", metrics=["accuracy"])
86     return model
87
```

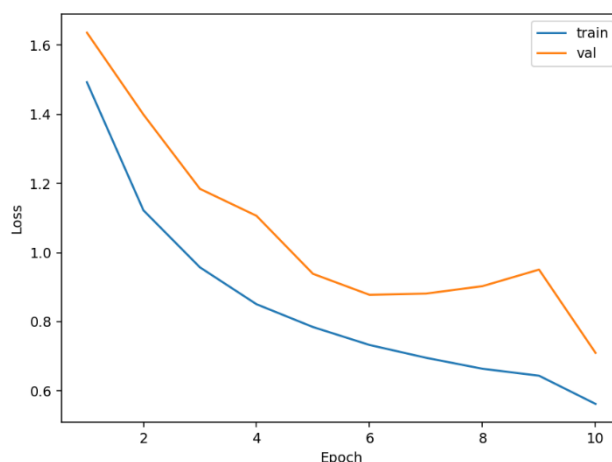
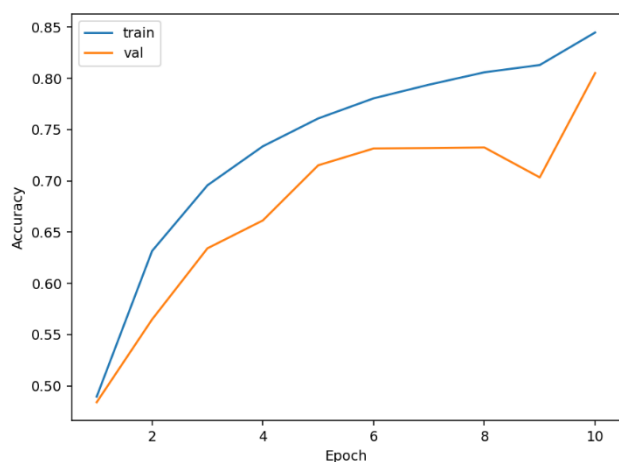
Designed Parameter (the later discussion will be made according to this design):

Kernel	5*5
L2 normalization	1e-4
Dropout	0
Learning rate	1e-3
Stride	1

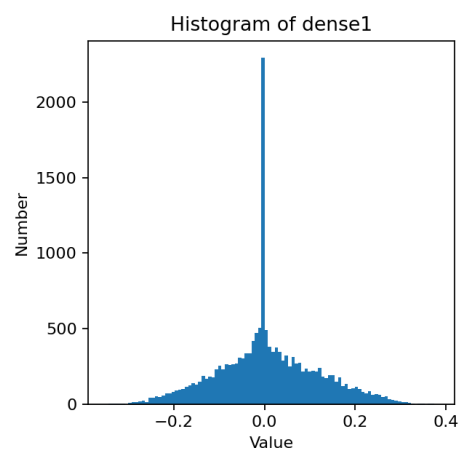
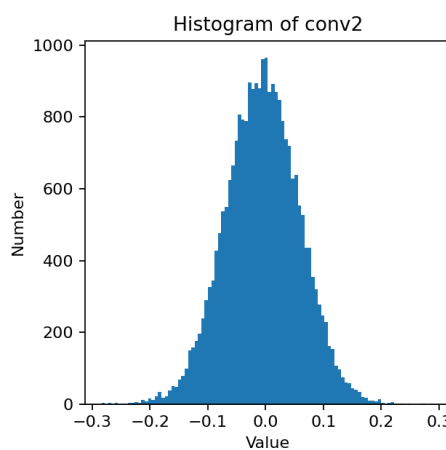
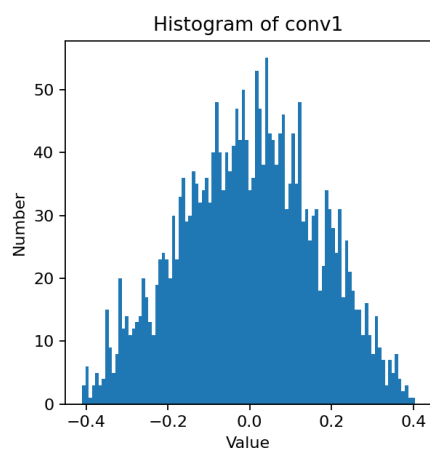
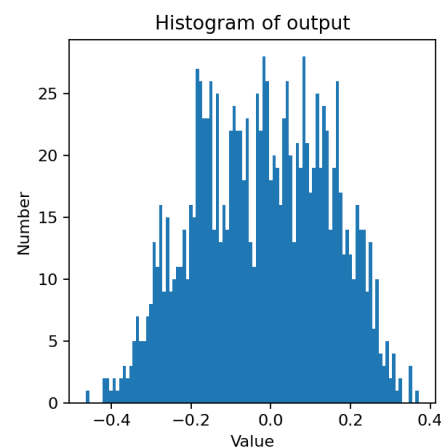
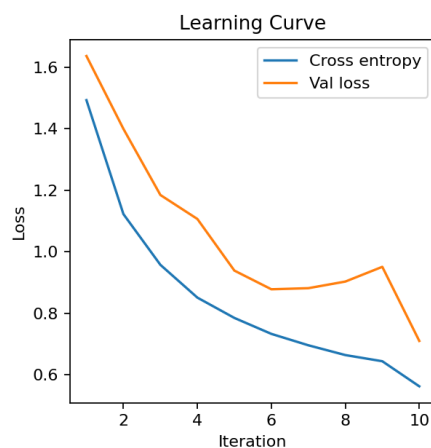
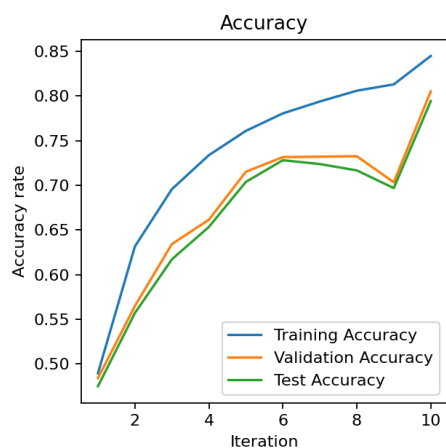
Results:

Test & Val Accuracy: 0.844 & 0.805

Test & Val Loss: 0.562 & 0.710



Accuracy, learning curve, and histograms of layers



2-1. Comparison of Different Stride Sizes and Filter Size

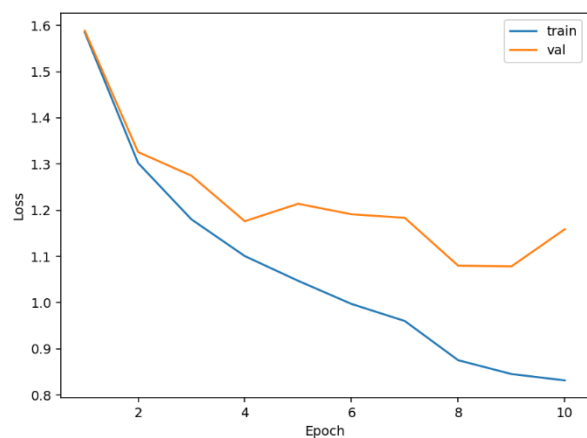
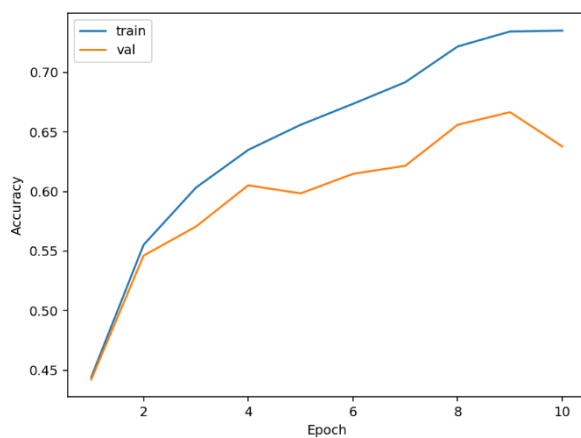
Kernel = 5*5, Stride = 2

(Stride = 2 only for the first layer, or the picture will be too small to calculate):

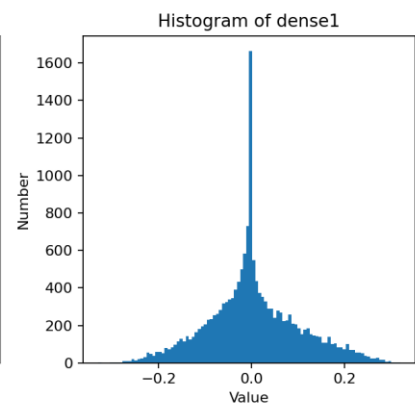
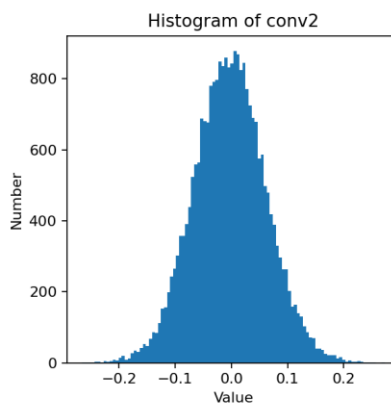
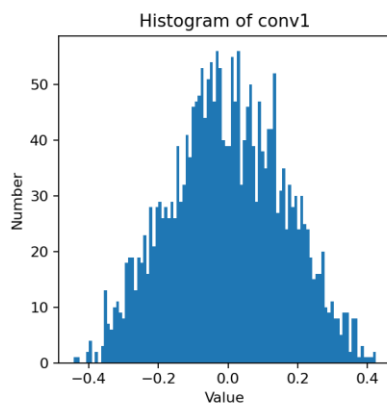
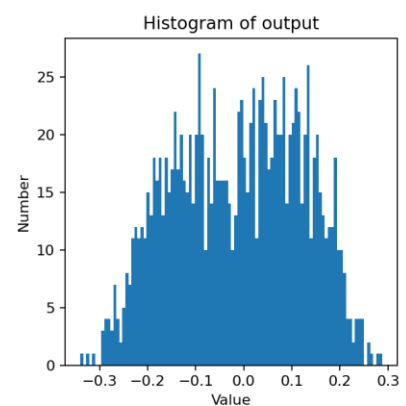
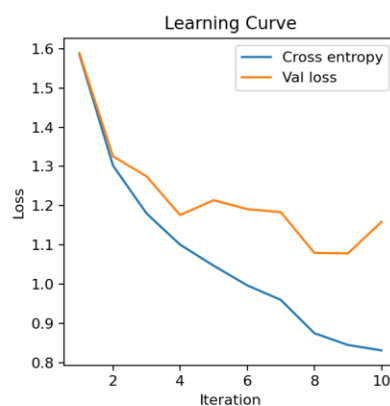
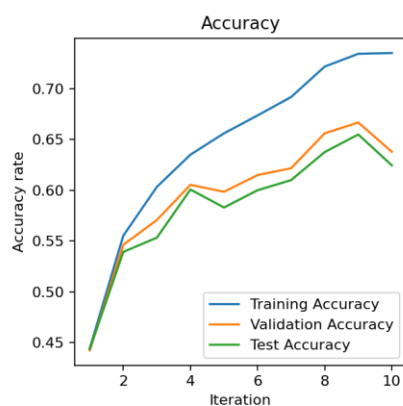
Kernel	5*5
L2 normalization	1e-4
Dropout	0
Learning rate	1e-3
Stride	2

Test & Val Accuracy: 0.735 & 0.6377

Test & Val Loss: 0.831 & 1.1589



Accuracy, learning curve, and histograms of the layer

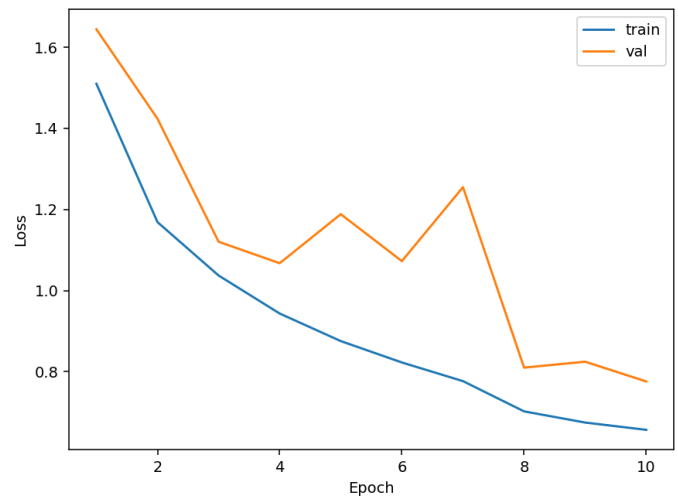
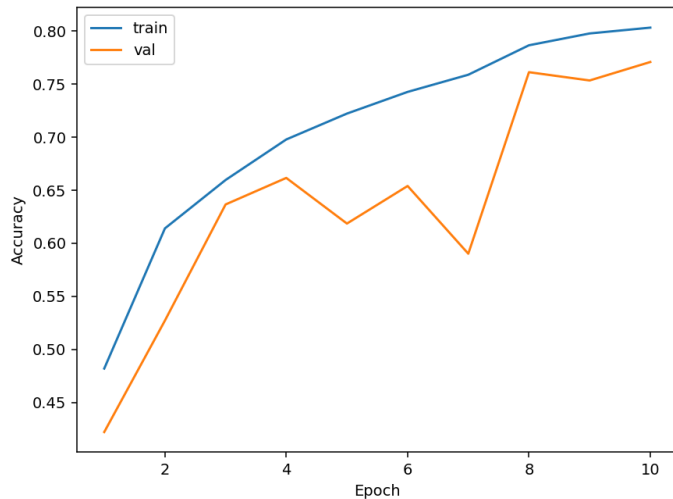


Kernel = 3*3, Stride = 1

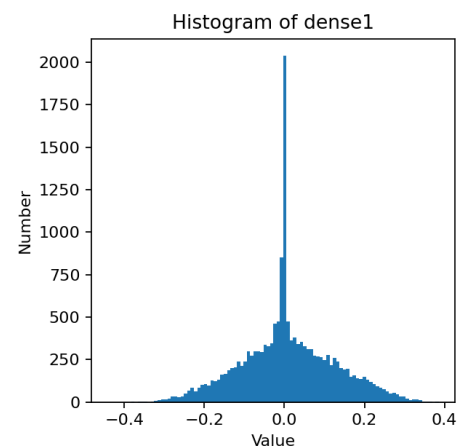
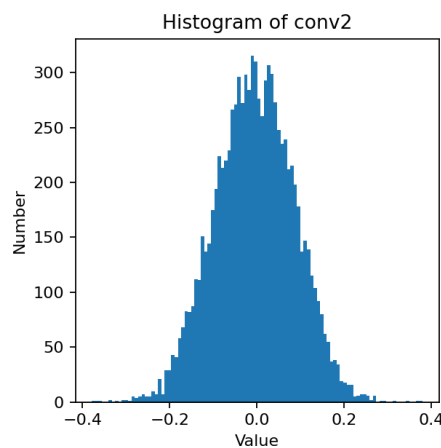
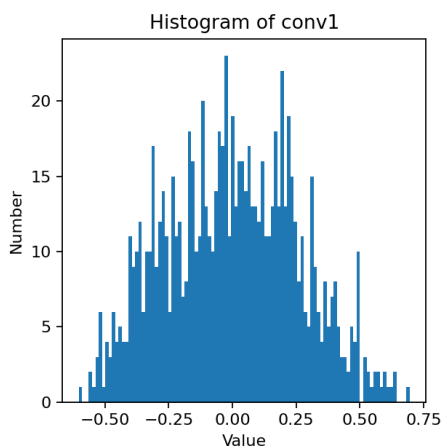
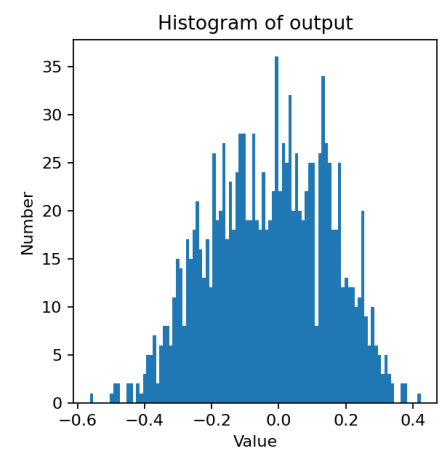
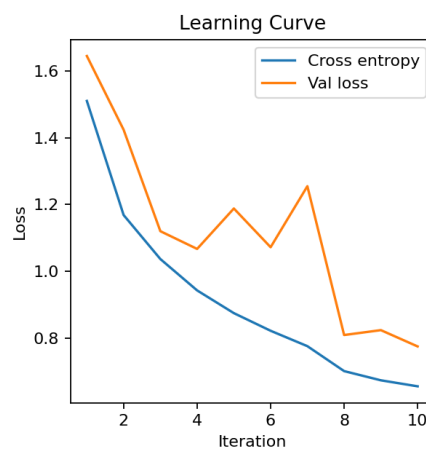
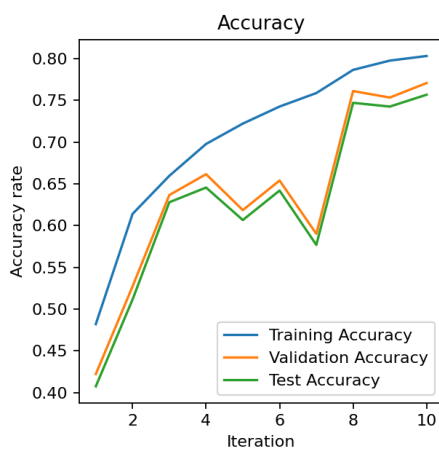
Kernel	3*3
L2 normalization	1e-4
Dropout	0
Learning rate	1e-3
Stride	1

Test & Val Accuracy:0.803 &0.770

Test & Val Loss: 0.656 & 0.776



Accuracy, learning curve, and histograms of the layer



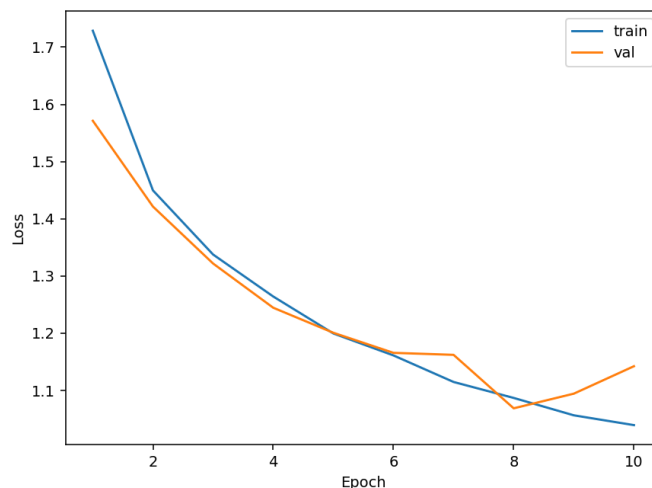
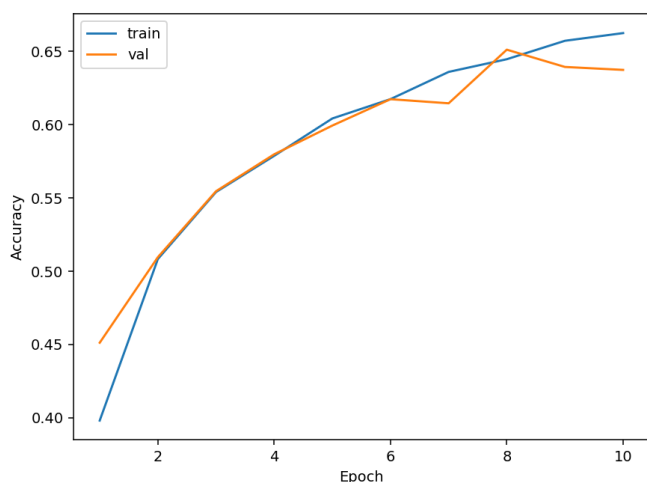
Kernel = 3*3, Stride = 2

(Stride = 2 only for the first layer, or the picture will be too small to calculate):

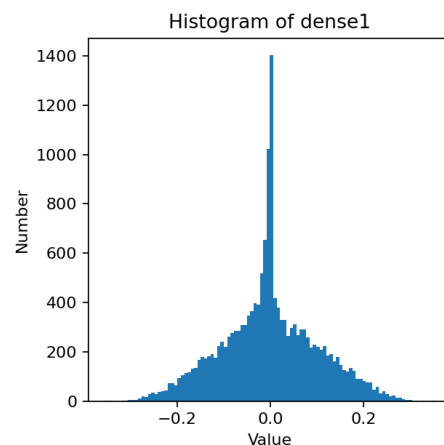
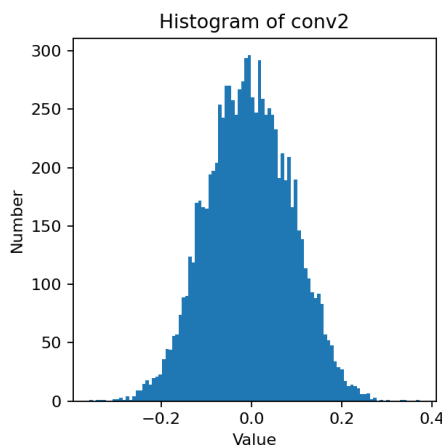
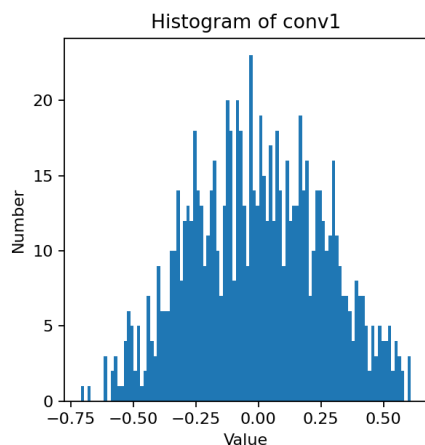
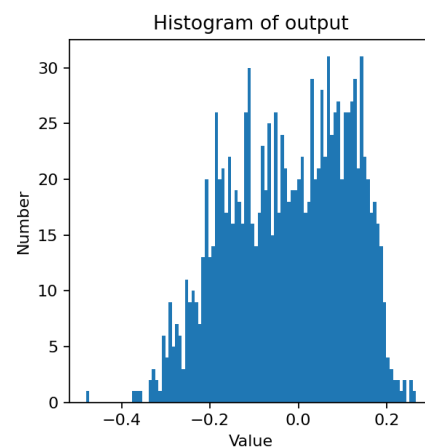
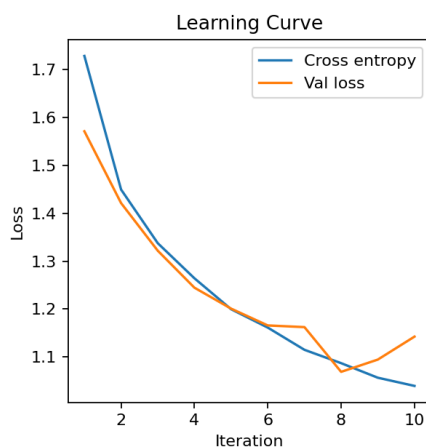
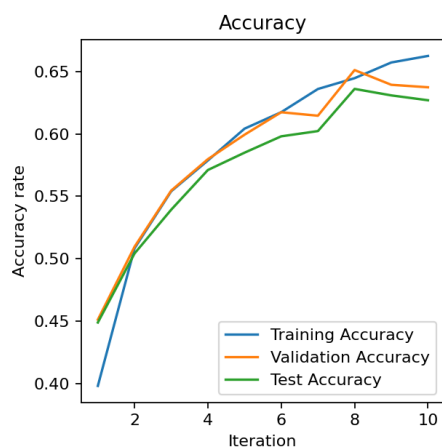
Kernel	3*3
L2 normalization	1e-4
Dropout	0
Learning rate	1e-3
Stride	2

Test & Val Accuracy: 0.663 & 0.637

Test & Val Loss:1.039 & 1.142



Accuracy, learning curve, and histograms of the layer



Analysis:

1. Effect of Filter (Kernel) Size

The filter size determines the receptive field of each convolutional layer:

- Larger filters (5×5) capture broader contextual information in a single operation, which can benefit small images such as CIFAR-10 (32×32 pixels).
- Smaller filters (3×3) focus on local details and are more parameter-efficient. When stacked in deeper architectures, multiple 3×3 convolutions can approximate the effect of a single 5×5 filter while introducing more nonlinearity.
- In this specific network with a moderate depth, the 5×5 filters perform slightly better because they extract spatial context more effectively within limited layers.

2. Effect of Stride

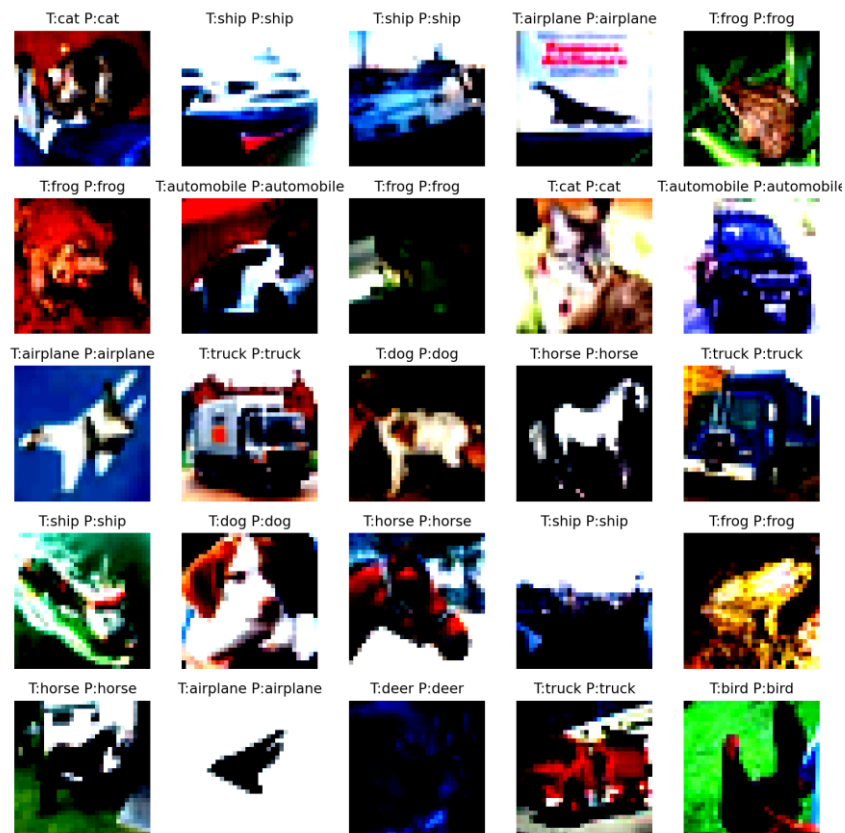
The stride controls the spatial downsampling rate:

- Stride = 1 preserves spatial resolution throughout the early layers, maintaining fine-grained information about edges and textures.
- Stride = 2 in the first layer immediately reduces the feature map size from 32×32 to 16×16 , causing a substantial loss of detail. Combined with subsequent MaxPooling layers, this results in excessive downsampling and a degradation in classification accuracy.
- Therefore, aggressive early downsampling (stride=2 in the first layer) negatively impacts performance, especially for small input images.

2-2. Examples of correctly classified and misclassified images

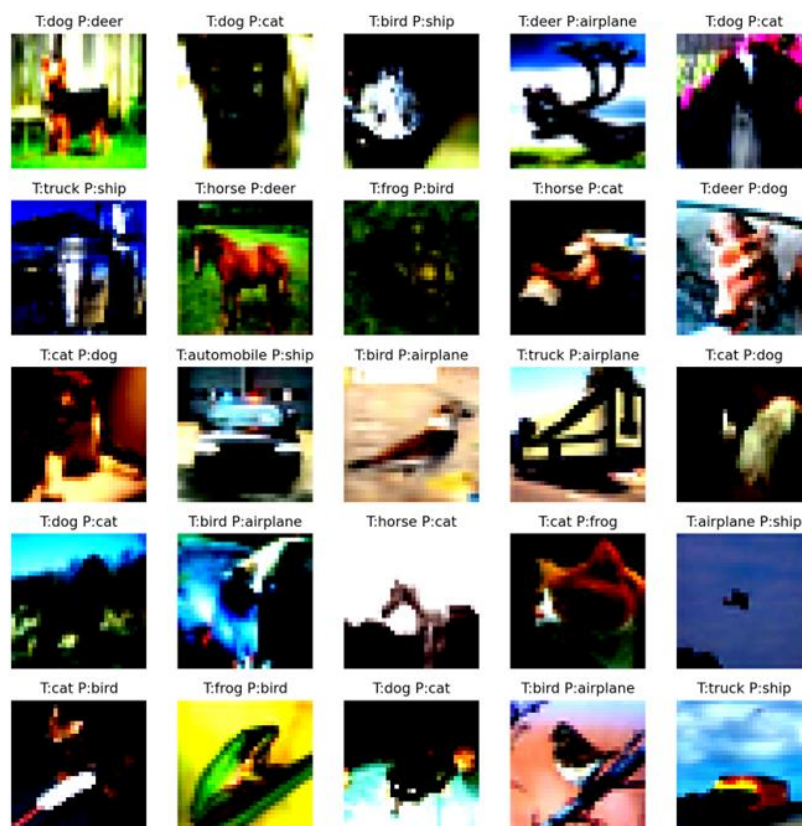
Correctly Classified Image:

The correctly predicted samples show the model can reliably capture global object structures and dominant color patterns.



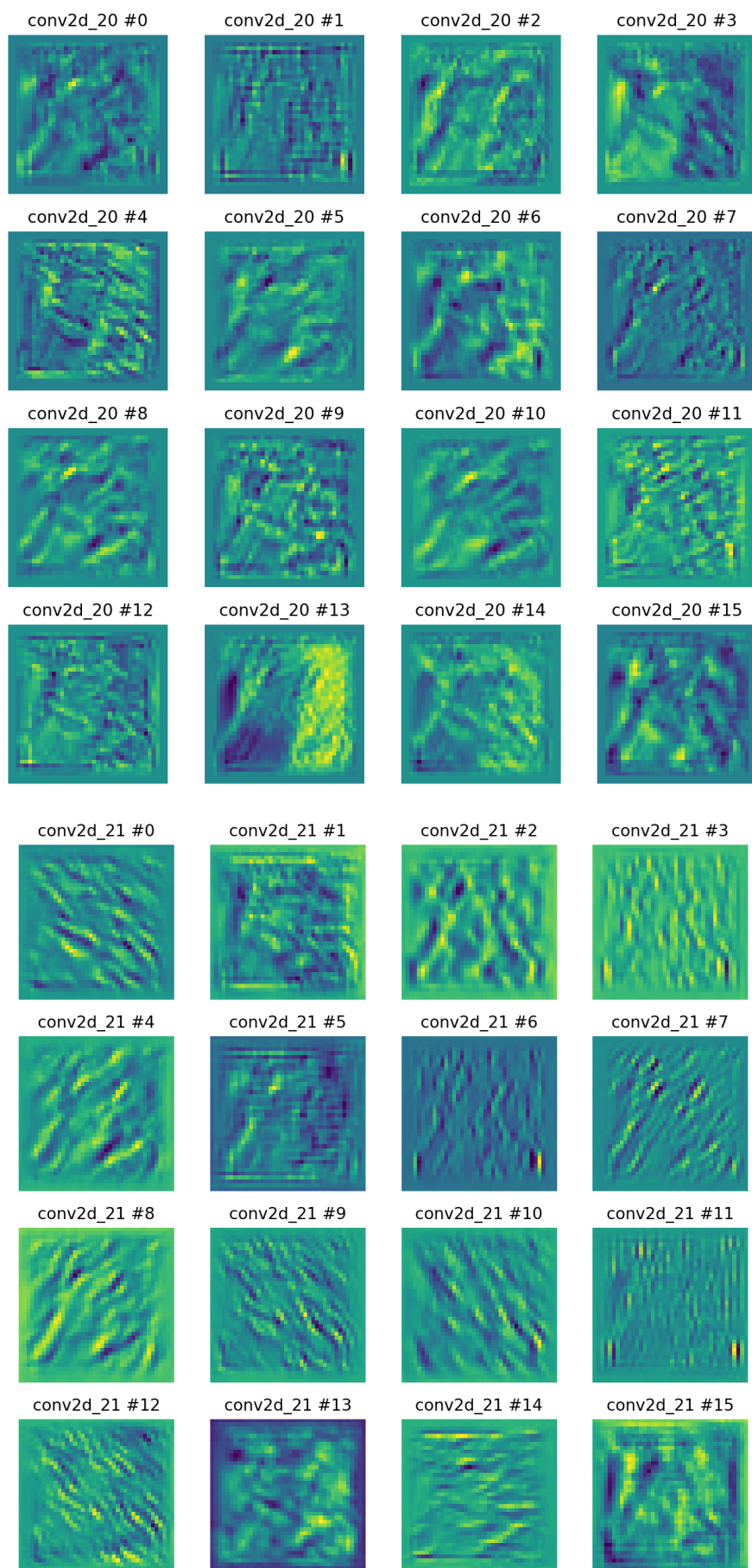
Misclassified Images:

The misclassified examples often contain ambiguous viewpoints, motion blur, or clutter in the



background.

2-3. Feature maps from different convolutional layers



Both layers exhibit structured activation patterns, indicating that the convolutional filters

effectively detect spatial features.

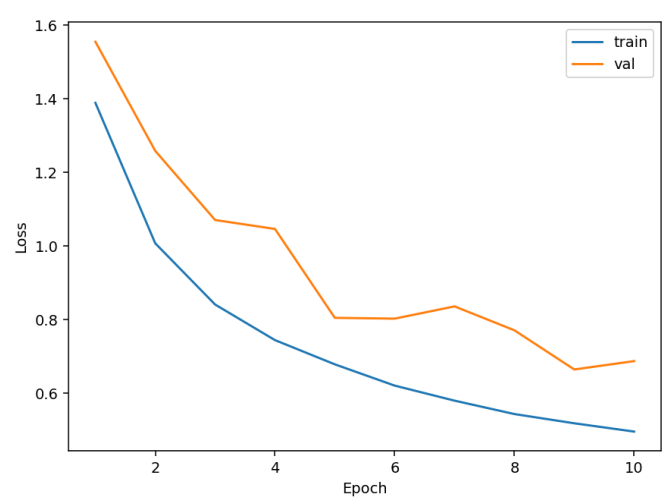
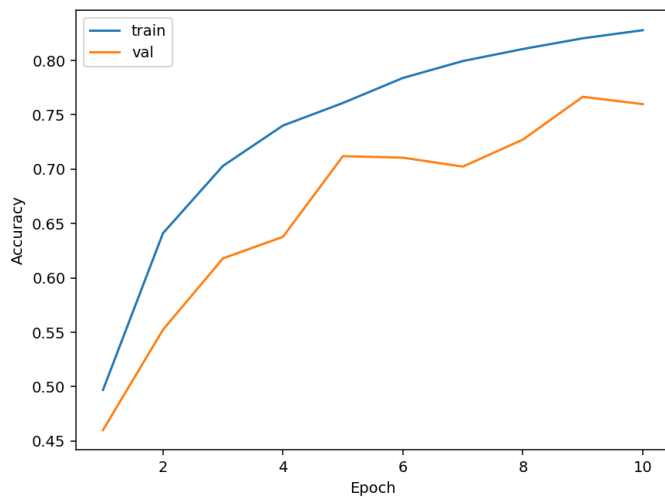
2-4. The effect of L2 regularization

Using L2 normalization =0 as a comparison of the designed NETWORK

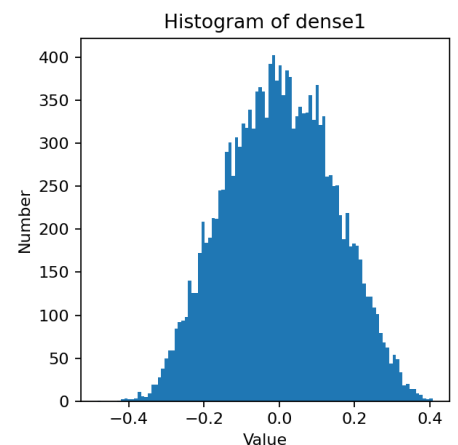
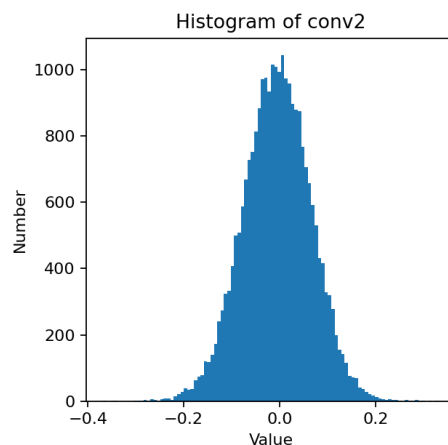
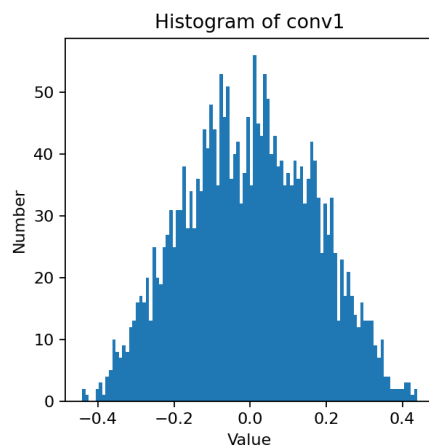
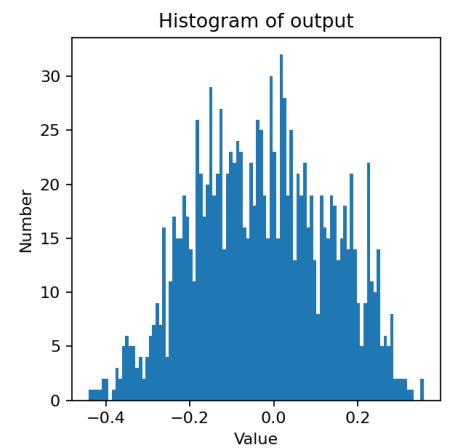
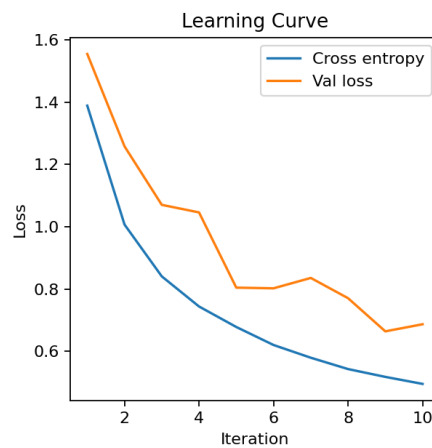
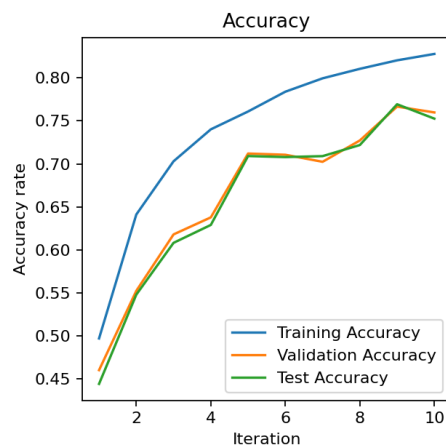
Kernel	5*5
L2 normalization	0
Dropout	0
Learning rate	1e-3
Stride	1

Test & Val Accuracy: 0.663 & 0.637

Test & Val Loss:1.039 & 1.142



Accuracy, learning curve, and histograms of the layer



Discussion about L2 Regularization

Condition	Test Accuracy	Validation Accuracy	Test Loss	Validation Loss
With L2 (1e-4)	0.844	0.805	0.562	0.710
Without L2 (0)	0.663	0.637	1.039	1.142

Learning Behavior

- Without L2 regularization, the training accuracy rises quickly, but the validation accuracy is relatively low. This indicates that the model overfits the training data, capturing noise and unnecessary patterns in the features.
- With L2 regularization, the learning curves are smoother, and the validation performance continues to improve. The gap between training and validation accuracy is smaller, suggesting enhanced model generalization.

Weight and Activation Distribution

- The weight histograms show that models trained with L2 have weight distributions that are narrower and more centered around zero, indicating better parameter control.
- In contrast, the non-regularized model exhibits broader distributions, implying the presence of large-magnitude weights that can lead to unstable or overfitted representations.
- The output activations of the L2 model are also more compact, reflecting more consistent and well-regularized feature responses across layers.

Interpretation

- L2 regularization (also known as weight decay) introduces a proportional penalty to the squared magnitude of the weights. This penalty discourages the use of extensive parameters and promotes smoother, more generalizable mappings.
- In this experiment, **L2 regularization substantially improved generalization performance** by reducing overfitting, stabilizing the training process, and preventing the model from memorizing the training set.

2-5. Preprocessing

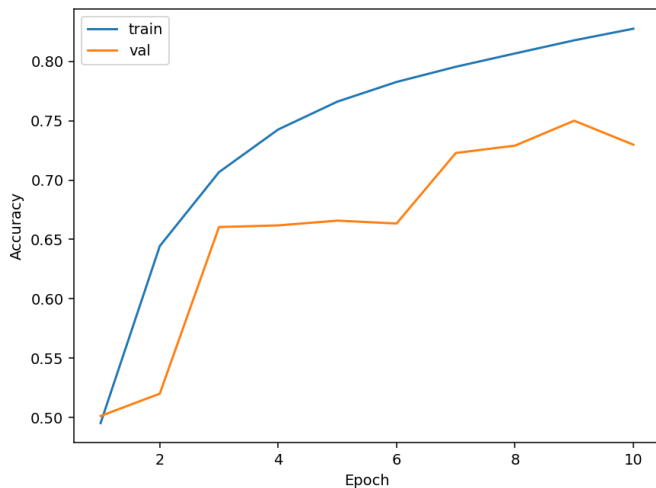
The additional program (scale to [0,1] and pre-channel standardization) and Setup

```
52 def preprocess_data(x_train, x_test):
53     """Return preprocessed data + stats used (mean/std)."""
54     x_train = x_train.astype("float32")
55     x_test = x_test.astype("float32")
56
57     x0_train = x_train.copy() # for visualization
58     x0_test = x_test.copy()
59
60     # scale to [0,1]
61     if PREPROC["scale01"]:
62         x_train /= 255.0
63         x_test /= 255.0
64
65     # per-channel standardization (using train statistics)
66     if PREPROC["per_channel_standardize"]:
67         mean = np.mean(x_train, axis=(0,1,2), keepdims=True)
68         std = np.std(x_train, axis=(0,1,2), keepdims=True) + PREPROC["eps"]
69         x_train = (x_train - mean) / std
70         x_test = (x_test - mean) / std
71     else:
72         mean = np.zeros((1,1,1,3), dtype=np.float32)
73         std = np.ones((1,1,1,3), dtype=np.float32)
74
75     stats = {
76         "scale01": PREPROC["scale01"],
77         "per_channel_standardize": PREPROC["per_channel_standardize"],
78         "mean": mean.squeeze().tolist(),
79         "std": std.squeeze().tolist()
80     }
81     return (x_train, x_test, stats, x0_train, x0_test)
```

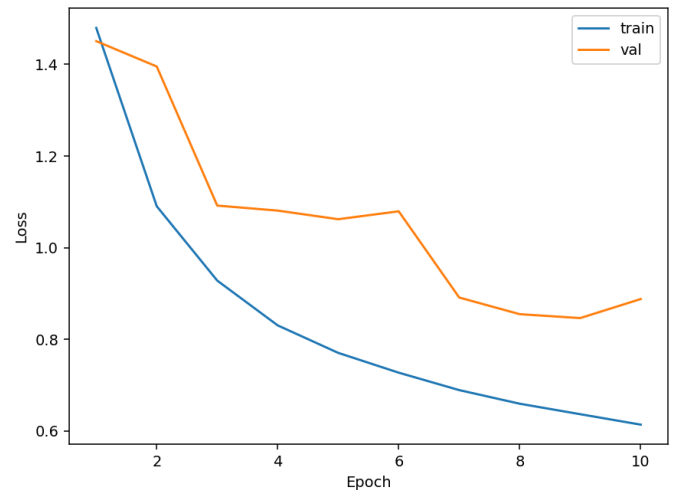
Kernel	5*5
L2 normalization	1e-4
Dropout	0
Learning rate	1e-3
Stride	1

Result:

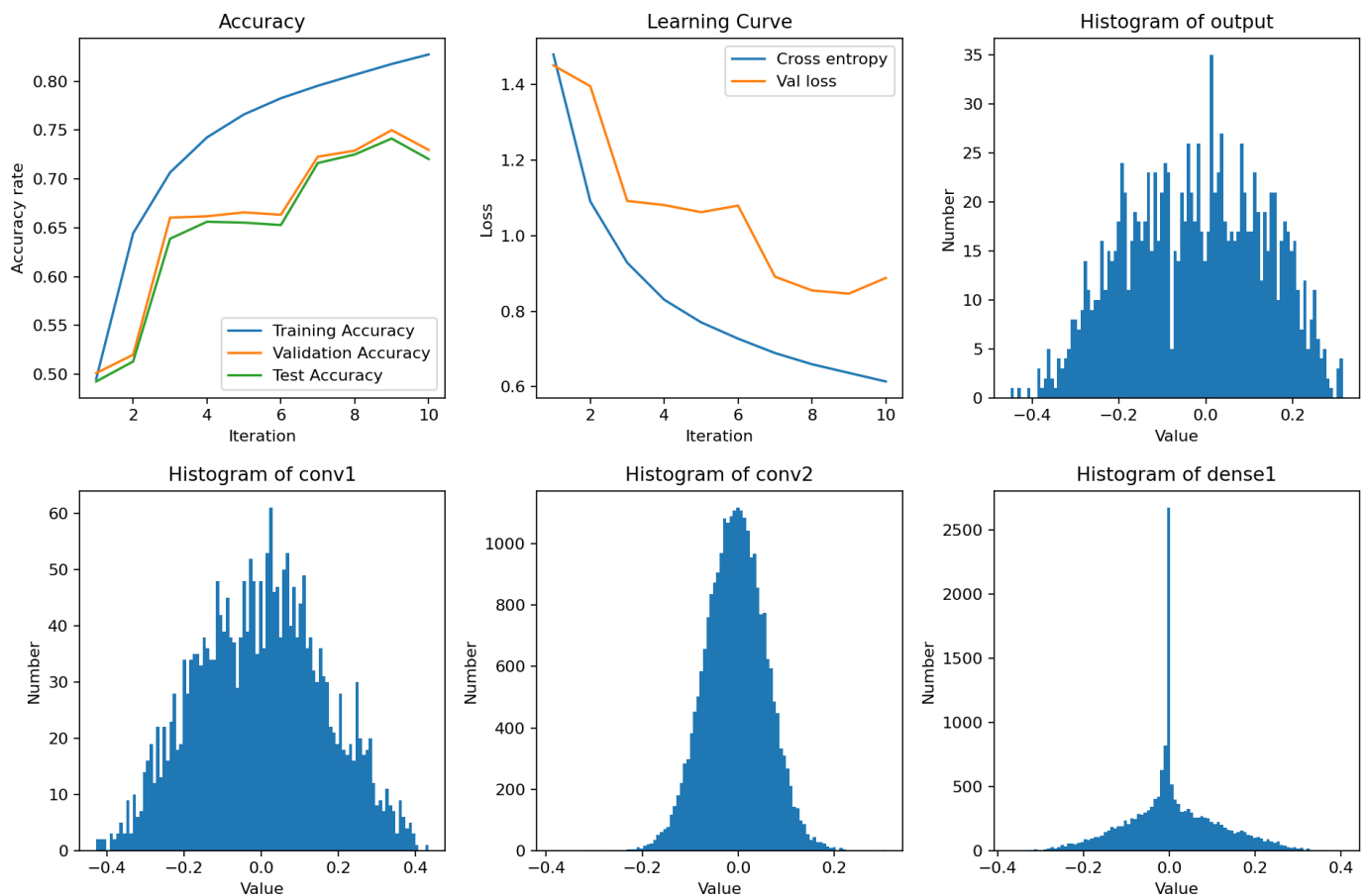
Test & Val Accuracy: 0.827 & 0.729



Test & Val Loss: 0.614 & 0.729



Accuracy, learning curve, and histograms of the layer



The model's performance did not improve. On the other hand, the sign of overfitting occurs. The validation accuracy was already constrained at epoch six and started to decline at epoch 9. The reason for this phenomenon is that in the original version of the program, the data were also normalized (L2 normalization). Not to mention that Batch Normalization is conducted after every convolution; as a result, most of the benefits of pre-channel standardization have been cancelled.